CSF205:
Web Service Development

Week 5: Parameter Passing and Responses

Swansea University
Prifysgol Abertawe

Department of Computer Science
Applied Software Engineering

Learning Outcomes



Lecture

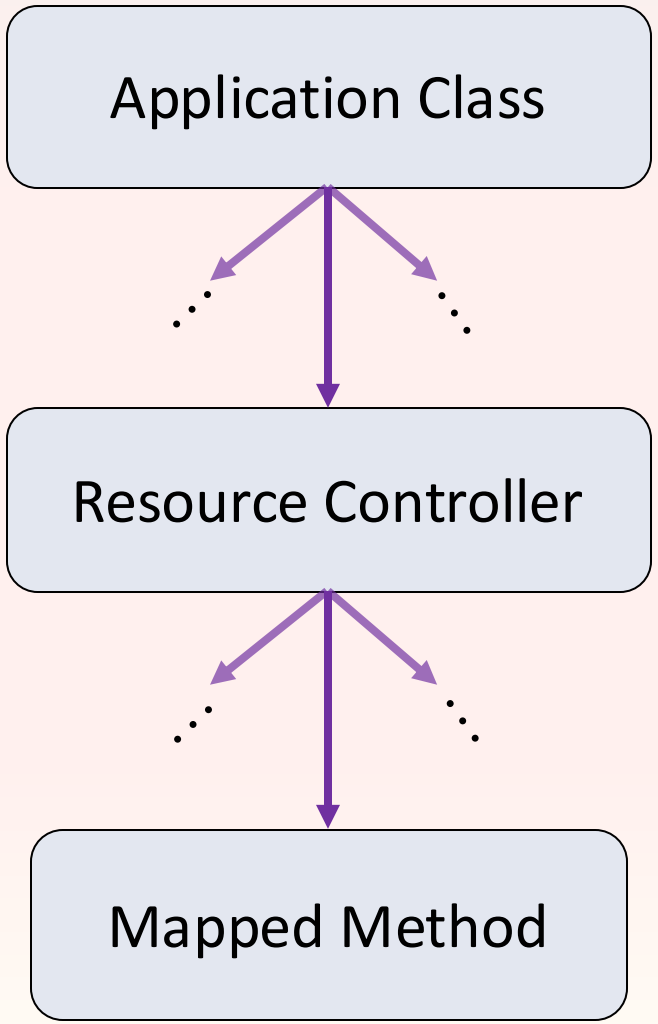
- Distinguish between different parameter passing methods in HTTP
- Use Spring Boot annotations to parse parameters
- Understand and evaluate when to use each parameter type
- Implement default values and optional parameters



Lab

- Interpreting and evaluating a service specification
- Implementing parameter passing mechanisms to query simple data
- Evaluating service operations and HTTP verbs

Introduction: Resource Method I/O



- Defines the web service endpoint root
- Can manage application configuration

- Groups services under related endpoints
- Contains HTTP-mapped Java methods
- Provides default configs for incoming requests

- Define mapping of HTTP method to Java Method
- Define sub-resource endpoints
- Implements service “business logic”
- Provides control over responses

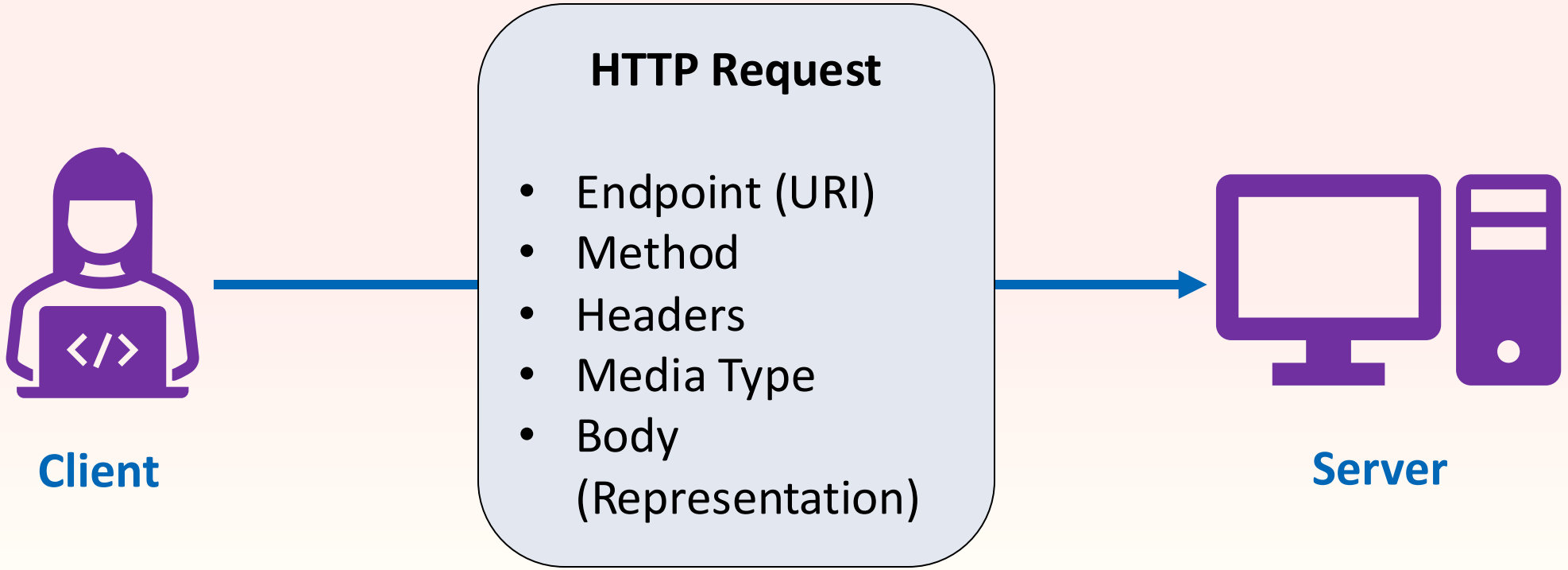
Parameter Passing: Customising Service Operations

A client sends a request to a web service. What does the request contain?

This data is used to fulfil the request. Is this information static?

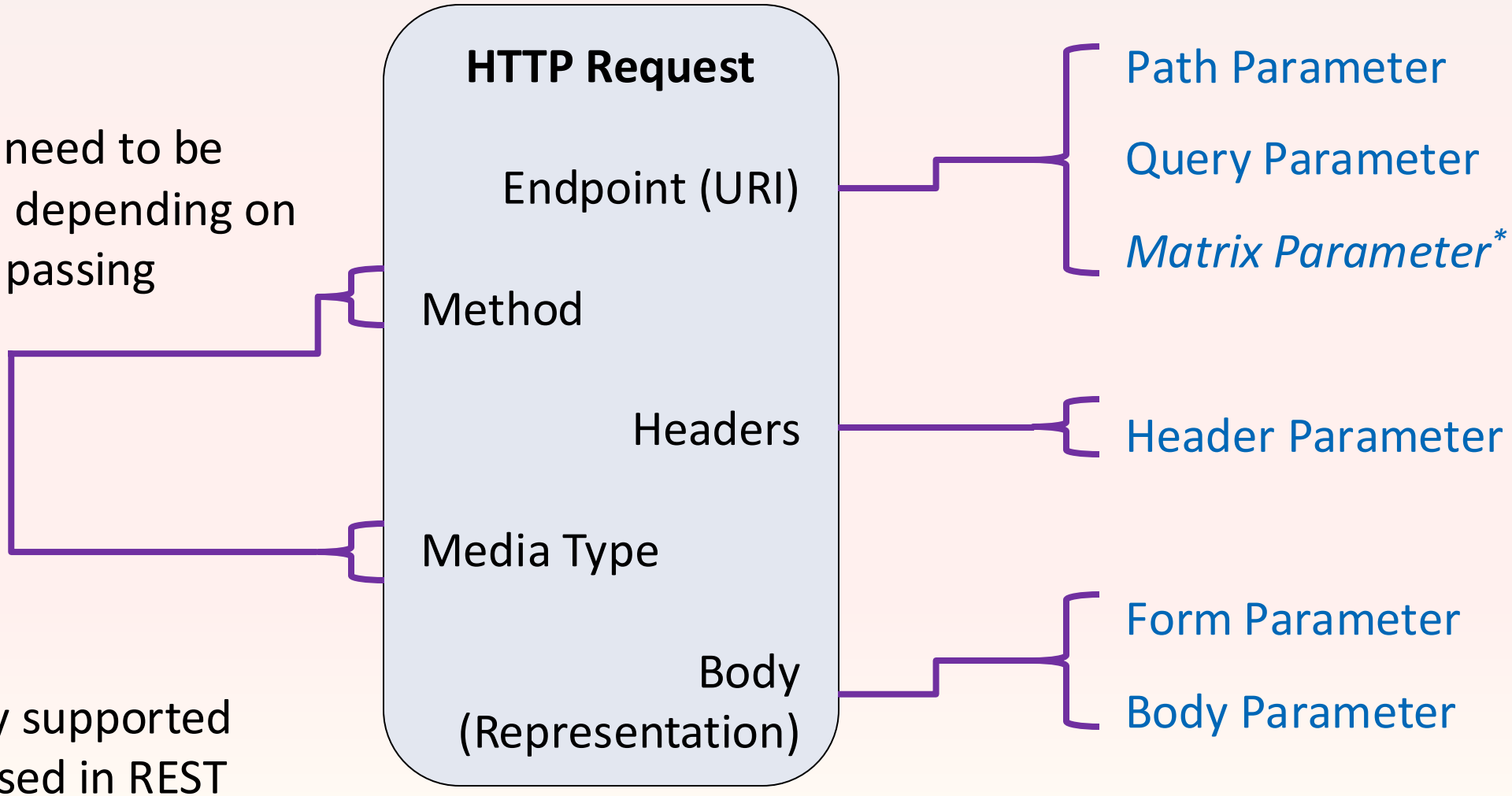
Part of the request describes the format of data and where it is sent.

Other parts can specify parameters that modify the service response.



Parameter Passing: Leveraging HTTP

Note:
These also need to be considered depending on parameter passing method!



* Not widely supported and rarely used in REST

Parameter Passing: Service Example

We have a service for documenting and querying mythical creatures.

The service allows clients to search, update, create and remove entries.

Those operations can be customised using parameters.



The root endpoint for this service will be:

arcanarchive/

Note:

The chosen method to pass parameters will influence the implementation

Parameter Passing: Query Parameter

One of the most common methods – you've likely seen it in your browser.

Customise request by placing key-value pairs in the URI of the form:

www.arcanarchive.me/creatures?type={option}&attribute={option}

www.arcanarchive.me/creatures?type=dragon&attribute=colour

www.arcanarchive.me/creatures?type=unicorn&attribute=magicPower

Parameter Passing: Path Parameters

Used to specify resources or sub-resources directly in the URI.

Allows the service to reuse a single endpoint to support multiple options:

www.archanaarcive.me/creatures/{type}/{attribute}

Client A could send a GET request for a specific resource:

www.archanaarcive.me/creatures/dragon/colour

Client B could send a GET request for a different resource:

www.archanaarcive.me/creatures/unicorn/magicPower

Parameter Passing: Header and Body Parameters

- Header parameters are Key: Value data stored in the HTTP header.
- Some headers have standard names, but you can also create your own!
- Body parameters are anything that is stored in the HTTP body.
- It can be plain text, xml, json or some more complex data

```
GET creatures/ HTTP/1.1
Host: www.arcanarchive.me
Content-Type: text/plain
Cookie: sessionID=secret12345
Custom-Header: someUsefulValue

Hello! Just some plain text here…
```

Parameter Passing: Form Parameter

- Uses same *key=value* syntax as Query Parameters.
- But they're passed in the HTTP request body, not in the URI.
- The Content-Type header defines the data format of the HTTP body.
- Thus, form parameters have their own specific Content-Type.

```
GET creatures/ HTTP/1.1
Host: www.arcanarchive.me
Content-Type: application/x-www-form-urlencoded


creatures?type=dragon&attribute=colour
```

DEMO

Implementing Parameter Passing in Spring Boot

Parameter Passing: General Conventions

Method	Use when	Avoid when
Path	• Referring to a specific resource • Service partitioned into hierarchy	• Completely optional filters • Arbitrary types of input
Query	• Optional filters on specific resources • Pagination or sorting	• Complex, structured data • Sensitive information
Form	• Supporting legacy HTML form submission	• Complex, structure data
Header	• Sending metadata (content types, etc.) • Sending authentication tokens	• Complex or application level data
Body	• Sending objects or complex data • Creating or updating resources	• Sending GET requests • Sending simple data for filters

PART 2

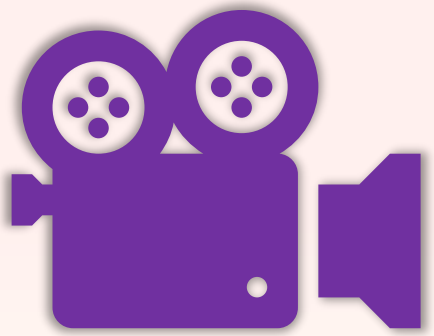
Media Types and Responses

Media Types: What's Happening?

Thus far, we haven't been coerced into configuring responses.

- Java Objects are automatically converted to JSON objects
- Plaintext magically appears in the HTTP response body
- Collections are automatically serialised into a list of JSON objects

… and vice versa!



How?

- Spring has been doing some heavy lifting for us.

Media Types: HTTP Headers

Central to data serialisation and deserialisation are the HTTP headers:

Content-Type

- Defines the media (MIME) type of the data in the HTTP body

Accept

- Defines the media (MIME) type the sender is capable of understanding

Media type definitions are straightforward: `type/subtype`

`text/plain`, `application/json`, `text/json`, `application/xml`,

etc…

Media Types: Resource Method

Assume a client request gets routed to the following resource method:

```
@GetMapping("/{index}")
public Iris getIrisByIndex(@PathVariable int index) {
      return data.get(index);
}
```

How does the client get:

- a structured JSON response?
- formatted data describing an Iris?

Media Types: “Automatic” Steps

```
@GetMapping("/{index}")
public Iris getIrisByIndex(@PathVariable int index) {
      return data.get(index);
}
```

Spring Boot:

1. Checks the media type in the Accept header of the incoming request
2. Finds an appropriate `HttpMessageConverter` for that media type
3. Deserialises the object using the Iris class getters and setters and a data-processing library called Jackson

DEMO

Media Types and Objects in Spring Boot

Responses: Beyond the Default

So, we can define how:

- Parameters are passed by the client to the service
- HTTP bodies can be constructed and interpreted when using objects
- Errors can be thrown and automatically return valid HTTP responses

- But what if we want to customise responses beyond that?
- Why haven’t we needed to manually set HTTP status codes?
- Why do all thrown exceptions look the same to the client?

Responses: Customising

Spring supports the `ResponseEntity<T>` class.

Allows us to build HTTP responses in segments, customising:

- Headers
- Response Codes
- Body



Wrapping the data that would have been automatically returned.

DEMO

The ResponseEntity in Spring Boot